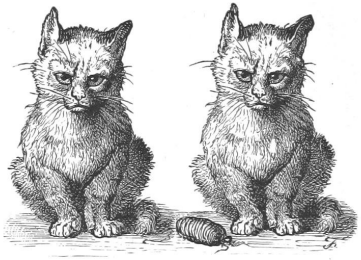
FRED TALKS

28th April 2026

CONTENTS

Mike Acton's Expectations of Professional Software Engineers

ADAM JOHNSON · 26 APR 2026 · SOURCE



grumpy cats ranting

In a 2019 talk/rant titled "Everyone Watching This Is Fired", games industry veteran Mike Acton rattled off a sample of 50 things he expects of developers he works with. The title refers to his tongue-in-cheek suggestion that anyone who doesn't meet all these requirements would be immediately fired.

Although his sense of humour isn't for everyone, the suggestions are valuable. The list forms a baseline for software engineers to compare themselves against, and it's not very specific to the games industry.

I couldn't find a textual version, so I've written up the list below, with my own expansions of each point and some further quotes from Mike. I hope this list inspires you to improve as an engineer, as it has me.

1. **I can articulate precisely what problem I am trying to solve.**

   It's all too easy to get stuck in the weeds and lose track of why you're doing what you're doing. Keep top of mind what the actual end goal is do, and you might spot an alternative path.

2. **I have articulated precisely what problem I am trying to solve.**

   Communicate the problem "out loud" to other team members, your product manager, etc.

3. **I have confirmed that someone else can articulate what problem I am trying to solve.**

Communication! Ensure your team is all on the same page. Make sure your understanding of the problem is complete.

4. **I can articulate why my problem is important to solve.**

   If you solve the problem you're working on, who benefits, and how much?

5. **I can articulate how much my problem is worth solving.**

   If you say it's worth "as long as it takes"… Mike does not have friendly words for you. For *any* problem there's a maximum amount of time and effort worth investing in solving it. At least have some idea of the upper bound.

6. **I have a Plan B in case my solution to my current problem doesn't work.**

   Imagine you're days or hours before the deadline, and you can tell that completing Plan A will be impossible. What do you do instead? Maybe you have a simplified algorithm, or you can disable a certain subsystem. Have more than one plan.

7. **I have already implemented my Plan B in case my solution to my current problem doesn't work.**

   Mitigate risk by writing the backup version first. This means you always have a safety net and you can learn more about the problem space in order to iterate on Plan A.

8. **I can articulate the steps required to solve my current problem.**

   Programming only works when you break down problems into manageable chunks. Have a sketch of the steps to the end state before you begin.

9. **I can clearly articulate unknowns and risks associated with my current problem.**

   There are always going to be things you don't know. You should know where they are in your plan, so you can manage them.

10. **I have not thought or said "I can just make up the time" without immediately talking to someone.**

    Say it's Wednesday, you have a project due on Friday, and you get some new task dropped on your lap. You think "I'll do the new thing now, and make up the time for the original task by Friday"… mistake! Communicate about the conflict on Wednesday. Your product manager will help manage the timing and risk.

11. **I write a “framework” and have used it multiple times to actually solve a problem it was intended to solve.**

    If you’re writing a tool of some kind, you should verify it works in practice. Too often people create something in isolation and it doesn’t end up delivering in the real world.

    (This is how Django came to be: from a real team making real websites, on deadlines!)

12. **I can articulate what the test for completion of my current problem is.**

    If you don’t know when to stop, you might find yourself going down rabbit holes, chasing unimportant marginal gains.

13. **I can articulate the hypothesis related to my problem and how I could falsify it.**

    If a hypothesis cannot be proven wrong, there’s no knowledge to be gained. As Karl Popper showed, science only works through falsification.

14. **I can articulate the (various) latency requirements for my current problem.**

    Any time you write code, you should consider when the output data is required. Not every caller needs output data instantly, and nor do you have an unbounded amount of time to perform everything. At least get an idea of the sensible bounds.

15. **I can articulate the (various) throughput requirements for my current problem.**

    How much data needs to come through the system? How many bytes, requests, or frames per second?

16. **I can articulate the most common concrete use case of the system I am developing.**

    You should know what actual users of your system will actually be doing most of the time. Having a vague idea doesn’t help, since knowing which pattersn are common informs which way to write a given piece of code.

17. **I know the most common actual, real-life values of the data I am transforming.**

    Beyond use cases, you should know the data inside the system. For example, if your function works with integers, you’d probably write it quite differently if 99% of the values are 0.

18. **I know the acceptable ranges of values of all the data I am transforming.**

Computer systems always have limits. Know the ranges for the data types you're working with (and enforce them).

19. **I can articulate what will happen when (somehow) data outside that range enters the system.**

    Murphy's Law says "anything that can go wrong will go wrong". Know how your system will behave in such cases, and handle such problems if necessary.

20. **I can articulate a list of input data into my system roughly sorted by likelihood.**

    Have an idea of the space of possible data, what's most likely, second most likely, etc. Code appropriately, for example checking for common error conditions first.

21. **I know the frequency of change of the actual, real-life values of the data I am transforming.**

    Reason about the frequency of change and figure out how often you'll want to calculate derived values.

22. **I have (at least partially) read the (available) documentation for the hardware, platform, and tools I use most commonly.**

    Read the friendly manual! Go a step beyond day-to-day reference, and try reading the full documentation to gain a deep understanding.

    (Jens Oliver Meiert calls reading the HTML specification the Web Developer's Pilgrimage.)

23. **I have sat and watched an actual user of my system.**

    Watching users can massively break shift your view of how your software works. Do it!

24. **I know the slowest part of the users of my system's workflow with high confidence.**

    Any workflow has a bottleneck. Make sure you know what it is so you can focus efforts there, if need be.

25. **I know what information users of my system will need to make effective use of the solution.**

    Think about what documentation or data users need to understand and use your solution.

26. **I can articulate the finite set of hardware I am designing my solution to work for.**

    Software requires hardware. Know what hardware your program targets, such as:

    - CPU architectures
    - Minimum requirements for memory, CPU speed, and network bandwidth
    - Input devices
    - Output devices
    - The environment the hardware runs in (e.g. data centre or living room)

27. **I can articulate how that set of hardware specifically affects the design of my system.**

    If you're targetting low end devices, how do you ensure you don't exhaust memory? If some users don't have pointing devices, how do you accommodate them?

28. **I have recently profiled the performance of my system.**

    If you're developing a local app, run profiling tools regularly to gain an idea of performance over time. With server based programs, you can install an APM (Application Performance Monitoring) tool in production and have continuous profiling data.

29. **I have recently profiled memory usage of my system.**

    Make sure you aren't wasting memory.

30. **I have used multiple different profiling methods to measure the performance of my system.**

    There's no perfect profiling tool, so know how to use more than one.

    For example, some great Python profilers are cProfile, py-spy, Austin, Scalene, Fil, and memray. They all have different characteristics and complement each other.

31. **I know how to significantly improve the performance of my system without changing the input/output interface of the system.**

Do you know the next step to optimize your system? You don't have to do it right now, as it may not be worth it, but you should have an idea what you'd do next to make your code faster. For example, use a faster but less convenient data structure, or convert a hot function into a faster language (such as Python to C with Cython).

This should also guide you to designing interfaces that are optimizable in the first place. For example, don't commit to returning expensive-to-compute results immediately, but instead return a promise.

32. **I know specifically how I can and will debug live release builds of my work when they fail.**

    Bugs are inevitable. You should know the tools that will let you work through those problems in production, such as logs, debuggers, or a live shell.

33. **I know what data I am reading as part of my solution and where it comes from.**

    Know where the data comes from, in what format, and how you can read it.

34. **I know how often I am reading data I do not need as part of my solution.**

    Data access is rarely optimal. You'll often be moving data that's not required for your solution, such as unnecessary fields or wrapper objects. If you don't know about this waste, you can't reason about whether it's worth the overhead or not.

35. **I know what data I am writing as part of my solution and where it is used.**

    All output data is intended for use by a human or another program. Be organized enough to know what the downstream consumers of your output are.

36. **I know how often I am writing data I do not need to as part of my solution.**

    Data output is also rarely optimal. Are you frequently writing out data that hasn't changed? Are you writing many fields when only one is used downstream? Again, know about it so you can reason about it.

37. **I can articulate how all the data I use is laid out in memory.**

    Many programming languages and frameworks can handle memory for you, but that doesn't abdicate you of responsibility. Know how your tools lay out memory, so you can tell when another approach makes sense.

For example, in Python most objects are based on dictionaries, so you should have a solid understanding of how they work, and alternatives like slotted classes or arrays.

38. **I never use the phrase "platform independent" when referring to my work.**

    Any system depends on many things below it. Know what they are.

39. **I never use the phrase "future proof" when referring to my work.**

    Future-proofing is "100% a fool's errand". "You can't pre-solve problems you have no information of."

40. **I can schedule my own time well.**

    "You're an adult person, just use a calendar."

41. **I am vigilant about not wasting others' time.**

    Don't waste time asking questions that you can google in five seconds. But also don't waste loads of time struggling for days alone when you could get help from a team member in minutes! Find the balance.

42. **I actively seek constructive feedback and take it seriously.**

    Ask for feedback and do something about it.

43. **I am not actively avoiding any uncomfortable (professional) conversations.**

    If there's something wrong at work, don't put off talking about it.

44. **I am not actively avoiding any (professional) conflicts.**

    If you've noticed something is going wrong, whether technically or communication wise, get those conflicts out in the open. Letting them stew never helps.

45. **I consistently interact with other professionals, professionally.**

    Be courteous and professional! Mike jokes about setting an incredibly low bar: no yelling, no hitting, …

46. **I can articulate what I believe others should expect from me.**

Have a standard for yourself and be ready to tell your co-workers what it is.

47. **I do not require multiple reminders to respond to a request or complete work.**

    “Waiting for someone else to poke you is not an effective way to get your job done.”

48. **I pursue opportunities to return value to the commons (when appropriate).**

    All our work builds on top of the work of countless others. At some point, you’ll have opportunities to give back to the community at large. For example, talking at meetups, making open source contributions, or even just discussing topics with your team to boost everyone’s skills.

49. **I actively work to bring value to the people I work with.**

    You’re part of a team, so work to help them.

50. **I actively work to ensure under-represented voices are heard.**

    Don’t stand by leaving this to be someone else’s problem. Do something to make sure that minorities are heard. This might mean ensuring that the minority person at work gets a chance to speak, that your hiring process is unbiased, or that your website is accessible for users who rely on screen readers.

Fin

Let’s learn and grow together,

—Adam

One summary email a week, no spam, I pinky promise.

Related posts:

Scaling Managed Agents: Decoupling the brain from the hands

ANTHROPIC ENGINEERING BLOG · 24 APR 2026 · SOURCE

Get started with Claude Managed Agents by following our docs.

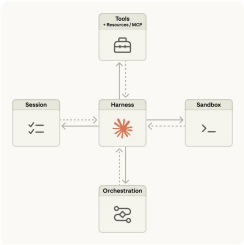
A running topic on the Engineering Blog is how to build effective agents and design harnesses for long-running work. A common thread across this work is that harnesses encode assumptions about what Claude can't do on its own. However, those assumptions need to be frequently questioned because they can go stale as models improve.

As just one example, in prior work we found that Claude Sonnet 4.5 would wrap up tasks prematurely as it sensed its context limit approaching—a behavior sometimes called "context anxiety." We addressed this by adding context resets to the harness. But when we used the same harness on Claude Opus 4.5, we found that the behavior was gone. The resets had become dead weight.

We expect harnesses to continue evolving. So we built Managed Agents: a hosted service in the Claude Platform that runs long-horizon agents on your behalf through a small set of interfaces meant to outlast any particular implementation—including the ones we run today.

Building Managed Agents meant solving an old problem in computing: how to design a system for "programs as yet unthought of." Decades ago, operating systems solved this problem by virtualizing hardware into abstractions—*process*, *file*—general enough for programs that didn't exist yet. The abstractions outlasted the hardware. The `read()` command is agnostic as to whether it's accessing a disk pack from the 1970s or a modern SSD. The abstractions on top stayed stable while the implementations underneath changed freely.

Managed Agents follow the same pattern. We virtualized the components of an agent: a session (the append-only log of everything that happened), a harness (the loop that calls Claude and routes Claude's tool calls to the relevant infrastructure), and a sandbox (an execution environment where Claude can run code and edit files). This allows the implementation of each to be swapped without disturbing the others. We're opinionated about the shape of these interfaces, not about what runs behind them.



We started by placing all agent components into a single container, which meant the session, agent harness, and sandbox all shared an environment. There were benefits to this approach, including that file edits are direct syscalls, and there were no service boundaries to design.

But by coupling everything into one container, we ran into an old infrastructure problem: we'd adopted a *pet*. In the pets-vs-cattle analogy, a pet is a named, hand-tended individual you can't afford to lose, while cattle are interchangeable. In our case, the server became that pet; if a container failed, the session was lost. If a container was unresponsive, we had to nurse it back to health.

Nursing containers meant debugging unresponsive stuck sessions. Our only window in was the WebSocket event stream, but that couldn't tell us *where* failures arose, which meant that a bug in the harness, a packet drop in the event stream, or a container going offline all presented the same. To figure out what went wrong, an engineer had to open a shell inside the container, but because that container often also held user data, that approach essentially meant we lacked the ability to debug.

A second issue was that the harness assumed that whatever Claude worked on lived in the container with it. When customers asked us to connect Claude to their virtual private cloud, they had to either peer their network with ours, or run our harness in their own environment. An assumption baked into the harness became a problem when we wanted to connect it to different infrastructure.

The solution we arrived at was to decouple what we thought of as the "brain" (Claude and its harness) from both the "hands" (sandboxes and tools that perform actions) and the "session" (the log of session events). Each became an interface that made few assumptions about the others, and each could fail or be replaced independently.

The harness leaves the container. Decoupling the brain from the hands meant the harness no longer lived inside the container. It called the container the way it called any other tool: `execute(name, input) → string`. The container became cattle. If the container died,

the harness caught the failure as a tool-call error and passed it back to Claude. If Claude decided to retry, a new container could be reinitialized with a standard recipe: `provision({resources})`. We no longer had to nurse failed containers back to health.

Recovering from harness failure. The harness also became cattle. Because the session log sits outside the harness, nothing in the harness needs to survive a crash. When one fails, a new one can be rebooted with `wake(sessionId)`, use `getSession(id)` to get back the event log, and resume from the last event. During the agent loop, the harness writes to the session with `emitEvent(id, event)` in order to keep a durable record of events.

Component	Interface (pseudocode)	Satisfied by

Session	`getSession(session_id) -> [Session, Event[]]`
	`getEvents(session_id) -> PendingEvent[] // not yet processed`
	`emitEvent(id, event)`
		Any append-only log that can be coalesced in order from any event point and accepts idempotent appends—Postgres, SQLite, in-memory array, etc.

Orchestration	`wake(session_id) -> void`	Any scheduler that can call a function with an ID and retry on failure—a cron job, a queue consumer, a while-loop, etc.
Harness	`yield Effect<T> -> EffectResult<T>`	Any loop that yields effects and appends progress to the Session.
Sandbox	`provision([resources]) execute(name, input) -> string`	Any executor that can be configured once and called many times as a tool—a local process, a remote container, etc.
Resources	`[{source_ref, mount_path}]`	Any durable store the container can fetch from by reference—Filestore, GCS, a git remote, S3.
Tools	`[name, description, input_schema]`	Any capability describable as a name and an input shape—MCP server, custom tool, etc.

The security boundary. In the coupled design, any untrusted code that Claude generated was run in the same container as credentials—so a prompt injection only had to convince Claude to read its own environment. Once an attacker has those tokens, they can spawn fresh, unrestricted sessions and delegate work to them. Narrow scoping is an obvious mitigation, but this encodes an assumption about what Claude can't do with a limited token—and Claude is getting increasingly smart. The structural fix was to make sure the tokens are never reachable from the sandbox where Claude's generated code runs.

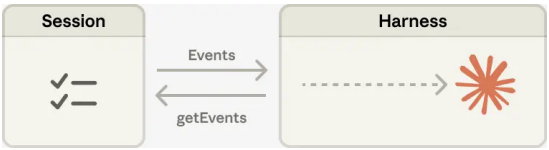
We used two patterns to ensure this. Auth can be bundled with a resource or held in a vault outside the sandbox. For Git, we use each repository's access token to clone the repo during sandbox initialization and wire it into the local git remote. Git `push` and `pull` work from inside the sandbox without the agent ever handling the token itself. For custom tools, we support MCP and store OAuth tokens in a secure vault. Claude calls MCP tools via a dedicated proxy; this proxy takes in a token associated with the session. The proxy can then fetch the corresponding credentials from the vault and make the call to the external service. The harness is never made aware of any credentials.

The session is not Claude's context window

Long-horizon tasks often exceed the length of Claude's context window, and the standard ways to address this all involve irreversible decisions about what to keep. We've explored these techniques in prior work on context engineering. For example, compaction lets Claude save a

summary of its context window and the memory tool lets Claude write context to files, enabling learning across sessions. This can be paired with context trimming, which selectively removes tokens such as old tool results or thinking blocks.

But irreversible decisions to selectively retain or discard context can lead to failures. It is difficult to know which tokens the future turns will need. If messages are transformed by a compaction step, the harness removes compacted messages from Claude's context window, and these are recoverable only if they are stored. Prior work has explored ways to address this by storing context as an object that lives *outside* the context window. For example, context can be an object in a REPL that the LLM programmatically accesses by writing code to filter or slice it.



In Managed Agents, the session provides this same benefit, serving as a context object that lives outside Claude's context window. But rather than be stored within the sandbox or REPL, context is durably stored in the session log. The interface, `getEvents()`, allows the brain to interrogate context by selecting positional slices of the event stream. The interface can be used flexibly, allowing the brain to pick up from wherever it last stopped reading, rewinding a few events before a specific moment to see the lead up, or rereading context before a specific action.

Any fetched events can also be transformed in the harness before being passed to Claude's context window. These transformations can be whatever the harness encodes, including context organization to achieve a high prompt cache hit rate and context engineering. We separated the concerns of recoverable context storage in the session and arbitrary context management in the harness because we can't predict what specific context engineering will be required in future models. The interfaces push that context management into the harness, and only guarantee that the session is durable and available for interrogation.

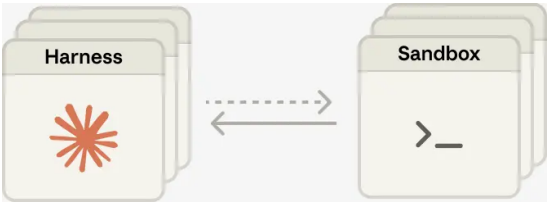
Many brains. Decoupling the brain from the hands solved one of our earliest customer complaints. When teams wanted Claude to work against resources in their own VPC, the only path was to peer their network with ours, because the container holding the harness assumed every resource sat next to it. Once the harness was no longer in the container, that assumption went away. The same change had a performance payoff. When we initially put the brain in a container, it meant that many brains required as many containers. For each brain, no inference could happen until that container was provisioned; every session paid the full container setup cost up front. Every session, even ones that would never touch the sandbox, had to clone the repo, boot the process, fetch pending events from our servers.

That dead time is expressed in time-to-first-token (TTFT), which measures how long a session waits between accepting work and producing its first response token. TTFT is the latency the user most acutely *feels*.

Decoupling the brain from the hands means that containers are provisioned by the brain via a tool call `(execute(name, input) → string)` only if they are needed. So a session that didn't need a container right away didn't wait for one. Inference could start as soon as the orchestration layer pulled pending events from the session log. Using this architecture, our p50 TTFT dropped roughly 60% and p95 dropped over 90%. Scaling to many brains just meant starting many stateless harnesses, and connecting them to hands only if needed.

Many hands. We also wanted the ability to connect each brain to many hands. In practice, this means Claude must reason about many execution environments and decide where to send work —a harder cognitive task than operating in a single shell. We started with the brain in a single container because earlier models weren't capable of this. As intelligence scaled, the single container became the limitation instead: when that container failed, we lost state for every hand that the brain was reaching into.

Decoupling the brain from the hands makes each hand a tool, `execute(name, input) → string`: a name and input go in, and a string is returned. That interface supports any custom tool, any MCP server, and our own tools. The harness doesn't know whether the sandbox is a container, a phone, or a Pokémon emulator. And because no hand is coupled to any brain, brains can pass hands to one another.



The challenge we faced is an old one: how to design a system for "programs as yet unthought of." Operating systems have lasted decades by virtualizing the hardware into abstractions general enough for programs that didn't exist yet. With Managed Agents, we aimed to design a system that accommodates future harnesses, sandboxes, or other components around Claude.

Managed Agents is a meta-harness in the same spirit, unopinionated about the *specific* harness that Claude will need in the future. Rather, it is a system with general interfaces that allow many different harnesses. For example, Claude Code is an excellent harness that we use widely across tasks. We've also shown that task-specific agent harnesses excel in narrow domains. Managed Agents can accommodate any of these, matching Claude's intelligence over time.

Meta-harness design means being opinionated about the interfaces around Claude: we expect that Claude will need the ability to manipulate state (the session) and perform computation (the sandbox). We also expect that Claude will require the ability to scale to many brains and many hands. We designed the interfaces so that these can be run reliably and securely over long time horizons. But we make no assumptions about the number or location of brains or hands that Claude will need.

Written by Lance Martin, Gabe Cemaj and Michael Cohen. Special thanks to the Agents API team and Jake Eaton for their contributions.

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · SOURCE

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments…). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs… but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding

doc saying “NEVER TOUCH STATE IN FOOBARDB DIRECTLY!”. You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool’s existence, or maybe it just preferred FooBarDB’s in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name “delete-everything”. Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

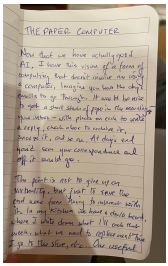
But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!

The paper computer

JAMES SOMERS · 19 APR 2026 · SOURCE

Now that we have actually good AI, I have this vision of a form of computing that doesn't involve me using a computer so much. Imagine you had the day's emails to go through. It would be nice if the ones that required a simple decision could be dispatched with a few pen-strokes: I could write down a date that would work for that meeting; check a box to accept that invitation; etc. If an email required me to review a draft, I'd love to mark up a print version on my couch, sans screen, and have those notes scanned and sent off as if I'd done the whole thing on Google Docs.

The point is not to give up on virtuality, but just to save the end user from having to interact with it. It's great to be able to send information to anyone in the world instantly; but let me do it without the glaring screen and the thousand distractions. Is such a thing even possible? I'm not sure, but I just now uploaded a first draft of this post, handwritten in a small notebook, into ChatGPT. Its transcription was nearly perfect.



A first draft of this blog post

Here's another example: when I really want to play with re-arrangements of a complicated outline, or when I want to collaborate with someone else on one, I find myself laying physical note cards on a table. Having real objects to work with allows for more flexibility than software. If the problem demands it, you can stack cards, draw on them, cut them, tape things to them, etc. —and none of these improvised ways of organizing has to be coded up in advance.

Space turns out to be a good way to organize information. I gather that in the paper days if you had a big project you were working on, say a book, it would spill out into the room you were working in: chapter outlines pinned up on the walls, stacks of books in meaningful piles on the floors, folders with drafts and clippings. Certain sections of the work in progress would become associated in your mind with certain parts of the room. Chapter 3 would be *over there*.

I rarely use paper or physical space this way because it lacks critical conveniences. A huge wall-mounted paper calendar is maybe the best way to plan and visualize a large coordinated effort. (In "making-of" documentaries you find that movie shoots are often planned this way.) Everyone can see it and point to it because it's at human-scale; you can express many

dimensions of information simultaneously using shape, color, position, size, and any other physical attribute. But I have a hard time understanding how I'd keep such a calendar up to date. In practice, like almost everyone else, I use a virtual calendar that automatically adds events as I'm invited to them, syncs with other people's calendars, reconciles time zones effortlessly, and interoperates with other programs like email.

Could we get the best of both worlds? In other words, shouldn't one goal of rapid technical advancement be some melding of the physical and virtual worlds such that I can sit quietly in an easy chair with pen and pad; or lay cards out on a table to organize my thoughts; or turn a room into the embodiment of a project; and yet have the same flexibility, portability, persistence, and remixability as in the digital versions of these things?

I spend a lot of my day on screens. There are many problems with these things, articulated well by Bret Victor in the context of his Dynamicland project. Screens are small, antisocial, and they have a tiny vocabulary of affordances compared to physical objects. Plus they have the problem that they make it difficult to *just* use your calendar, todo list, or map—or even just respond to a friend's message—without encountering something else along the way, like a social network, short-form video, Slack, the news, or some other notification. To state the obvious: your phone is the best place to keep your calendar and inbox and todo list because you always have it with you, but of course that makes it ripe for other intruders. Bundling makes your phone indispensable, but also a menace.

If nothing else I'd like it if operating systems and web browsers helped me be less distracted and frenetic, instead of encouraging exactly that multi-tasking freneticism. When I opened my phone or computer, it'd be nice if it was constrained to operate in a mode purpose-built for whatever task I intended to use it for. If I want to look something up, for instance, my phone should be a look-up machine (ie no texts, no apps, no ads, just a place for my question and the answer). If I want to compose a word of the day entry, I should launch straight into a browser with the tabs I regularly use for that, including the CRM, Webster's dictionary, and the OED; if I want to work on an article, my computer should assume the form of a typewriter, word processor, or McPhee mode note-processor depending on what stage I'm at. In each of these modes everything else should disappear, inaccessible. At least then you could mimic in software that thing you get from physical objects—which is that they are usually built to do one, and only one, thing well. My alarm clock, for instance, is just an alarm clock; and that's what I like about it!

Not too long ago I spoke to a roboticist for an article I was writing, who worked with large autonomous earth-moving machines—e.g. a retrofitted excavator that could lift a boulder, scan its every edge and dimple, then model how it would settle amongst other boulders in a retaining wall before placing it there. He imagined a future in which such machines enabled a return to natural materials in the built world. He talked about old stone walls he’d seen in New England.

Those walls, made of loose rocks found in situ, are lovely and sturdy, and adaptive—constantly rebuilt as farmers go about their work and notice areas that need patching up, adding stones they find lying around. But this is the very reason such walls aren't really built anymore. They're too labor-intensive. We live in a prefab world because the scarce thing now is not material or money but "the works and days of hands."

I am moved by the idea that our future could feel less futuristic than pastoral. High tech could save us from high tech. We'd go back to the old interfaces without giving up the conveniences of the new ones. Read, write, communicate, create—and hardly ever see or touch a screen.

I'm not sure that'll happen or be what people want. But shouldn't we be thinking of ways to use the new magic to spend less time tapping and clicking?